

Shortest Job First (SJF)

SJF chooses the shortest ready job whenever the CPU becomes free. Once chosen, the job runs to completion.

1. Model

The visualizer uses one CPU, whole ticks, and CPU-only processes. **Service time** is the total CPU work a process needs. The scheduler knows this value from the input. Running for one tick reduces remaining service by one; switching processes takes no time.

Only processes that have arrived are eligible. A process's arrival time does not change while it waits.

2. Scheduling rules

Rule 1. Choose the shortest ready service time

When the CPU is free, compare the original service times of all ready processes and select the smallest. The choice is made from the whole ready queue.

If B needs four ticks and C needs one, C is selected even if B has waited longer. In the visualizer, the queue stays in insertion order, so the chosen process may come from the middle or back.

Rule 2. Resolve equal lengths by arrival, then input order

Among jobs with equal service times, choose the earlier arrival. If arrival times also match, choose the one listed first in the input. Process IDs do not determine the order.

For example, if C arrives at time 1 and B at time 2, both needing two ticks, C takes precedence when both are ready.

Rule 3. Do not interrupt a running process

SJF is non-preemptive. A shorter process arriving later must wait for the current one to finish.

If A starts a five-tick job at time 0 and B arrives at time 1 needing one tick, A still runs until time 5. The scheduler compares ready jobs again only after A finishes.

Rule 4. Finish at zero remaining service

A process leaves after its final CPU tick. Dispatch may occur at the same boundary, with no extra tick for the switch.

Rule 5. Do not wait for a future short job

If any process is ready and the CPU is free, select a ready process now. Idle only when none is ready. Stop when all processes finish.

For example, A must start at time 0 if it is the only arrival, even when the input shows that a shorter job will arrive at time 1.

After accounting for the preceding tick:

1. Remove the running process if it has finished.
2. Admit all arrivals at the current time.
3. If the CPU is free, choose the shortest ready job, using Rule 2 for ties.

4. Worked example

<i>Process</i>	<i>Arrival</i>	<i>Service</i>
A	0	5
B	1	2
C	1	1

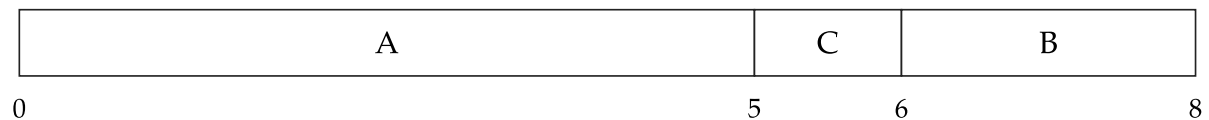


Figure 1. CPU schedule (time in ticks).

A is the only ready process at time 0, so it starts immediately. B and C arrive at time 1 and wait. At time 5, C is chosen because its one-tick service is shorter than B's two-tick service. B starts after C finishes at time 6.